ANALIZADOR SINTÁCTICO PASCAL – C

DOCUMENTACIÓN DEL ASIGNACIÓN ENTRE LOS INTEGRANTES

ALGORITMO FINAL DE ANÁLISIS SINTÁCTICO LR

Equipo 6

Integrantes:

Sánchez Aguilar Luis Ángel
Álvarez Olivera Irving Damián
Ramírez Manzano Jaret
García Barela Ángel Andrés
López Osorio Haniel
Martinez Cesar Zaid
Soriano Martínez Miguel Ángel

Materia: Compiladores

Profesor: David Martínez Torres

Fecha de entrega: Viernes 30 de octubre del 2026

UNIVERSIDAD TECNOLÓGICA DE LA MIXTECA

ÍNDICE

ÍNDICE **2**
1. Introducción **2**
2. Descripción general del sistema **3**
3. Objetivo del programa **3**
4. Arquitectura del sistema **3**
1. **4.1. Entrada de la gramática y la tira de tokens** 3
2. **4.2. Inicialización de la pila y del apuntador** 4
3. **4.3. Consulta de la tabla de análisis sintáctico** 4
4. **4.4. Ejecución de acciones de desplazamiento** 5
5. **4.5. Ejecución de acciones de reducción** 5
6. **4.6. Estados de aceptación y error** 6
7. **4.7. Organización de archivos y ejecución automática** 6
5. Tecnologías utilizadas **7**
6. Flujo de funcionamiento **7**
7. Conclusiones **8**
8. Asignaciones de la semana **8**
9. Evidencias **10**

1. Introducción

El análisis sintáctico es una de las etapas fundamentales de un compilador, ya que permite comprobar si una secuencia de tokens cumple con las reglas establecidas por una gramática. Después de construir la colección canónica y la tabla de análisis sintáctico, es necesario implementar un algoritmo que utilice la información generada para procesar una entrada y determinar si esta pertenece al lenguaje definido por la gramática.

En esta etapa del proyecto se desarrolló el algoritmo de análisis sintáctico LR, tomando como referencia el procedimiento proporcionado en clase. El algoritmo utiliza una pila, una tira de tokens y los datos de la tabla de análisis sintáctico para ejecutar desplazamientos y reducciones hasta llegar a un estado de aceptación o detectar un error.

Asimismo, se organizaron los componentes del programa en archivos separados y se incorporó una funcionalidad para facilitar las ejecuciones mediante la carga

automática de una gramática y una tira de tokens. Esto permite agilizar las pruebas y comprobar el funcionamiento del analizador sin tener que introducir manualmente todos los datos en cada ejecución.

2. Descripción general del sistema

El sistema corresponde a la etapa final del analizador sintáctico del proyecto. Su función principal consiste en utilizar la tabla de análisis sintáctico previamente generada para determinar si una tira de tokens puede ser reconocida de acuerdo con las producciones de una gramática.

Para realizar este procedimiento, el algoritmo mantiene una pila en la que se almacenan los símbolos y estados necesarios durante el análisis. También utiliza un apuntador que indica el token que se está procesando dentro de la entrada. En cada iteración se consulta la tabla mediante el estado situado en la parte superior de la pila y el token actual.

Dependiendo del resultado de la consulta, el algoritmo puede ejecutar una acción de desplazamiento, aplicar una reducción mediante una producción de la gramática, aceptar la entrada o señalar un error. El procedimiento continúa hasta que se alcanza una de las condiciones de terminación.

Además de la implementación del algoritmo, se crearon archivos destinados a separar su funcionamiento del conjunto de ejemplos utilizados durante las pruebas. También se incorporó una opción de ejecución automática que permite disponer de una gramática y una tira de tokens para realizar las comprobaciones de manera más rápida.

3. Objetivo del programa

Implementar el algoritmo de análisis sintáctico LR para procesar una tira de tokens mediante la tabla de análisis sintáctico, utilizando una pila para controlar los estados y símbolos durante la ejecución, hasta determinar si la entrada es aceptada o si se produce un error.

Como objetivos complementarios, se busca integrar el algoritmo con los procesos desarrollados anteriormente, organizar el código y los ejemplos en archivos independientes y facilitar las pruebas mediante la incorporación automática de una gramática y una tira de tokens durante la ejecución del programa.

4. Arquitectura del sistema

4.1. Entrada de la gramática y la tira de tokens

El proceso comienza con la disponibilidad de una gramática y una tira de tokens que se utilizarán durante el análisis. La gramática contiene las producciones que definen las estructuras válidas del lenguaje, mientras que la tira de tokens representa la entrada que se desea evaluar.

El algoritmo utiliza la tabla de análisis sintáctico construida previamente a partir de la gramática y de los estados de la colección canónica. De esta manera, no necesita construir nuevamente todos los estados durante cada operación de análisis, sino que consulta la información generada por las etapas anteriores.

La entrada debe incluir el marcador de fin de cadena $, de acuerdo con el procedimiento presentado en clase, para que el algoritmo pueda identificar el final de la tira de tokens y determinar si corresponde aceptar la entrada.

4.2. Inicialización de la pila y del apuntador

Una vez disponibles los datos necesarios, se inicializa la pila con el estado inicial, identificado como el estado 0. Posteriormente, se establece un apuntador que indica la posición del primer token de la tira de entrada.

El estado situado en la parte superior de la pila se obtiene mediante la operación stacktop(pila). Este estado se utiliza junto con el token actual para consultar la tabla de acciones del analizador sintáctico.

Esta inicialización permite que el algoritmo comience el procesamiento de la entrada desde el estado inicial definido por la colección canónica.

4.3. Consulta de la tabla de análisis sintáctico

En cada iteración, el algoritmo obtiene el estado actual de la pila y el token señalado por el apuntador. Con ambos datos consulta la tabla de acciones mediante la expresión:

Accion[s, a]

Donde:

- **s:** estado situado en la parte superior de la pila.
- **a:** token actual de la tira de entrada.
- **Accion:** sección de la tabla que determina la operación que debe ejecutarse.

El resultado de esta consulta permite identificar la acción correspondiente. Dependiendo de la tabla, puede tratarse de un desplazamiento, una reducción, una aceptación o una condición de error.

Este mecanismo constituye la base del procedimiento, ya que permite ejecutar las decisiones previamente determinadas por la tabla de análisis sintáctico.

4.4. Ejecución de acciones de desplazamiento

Cuando la tabla indica una acción de desplazamiento, representada como d_j, el algoritmo realiza las siguientes operaciones:

1. Imprime o registra el estado correspondiente a la acción, según la implementación.
2. Introduce el token actual en la pila.
3. Introduce el estado de destino j en la pila.
4. Avanza el apuntador para procesar el siguiente token de la entrada.

El desplazamiento permite consumir progresivamente los tokens y avanzar entre los estados del autómata de análisis sintáctico.

La pila almacena tanto los símbolos como los estados asociados, por lo que las operaciones posteriores pueden recuperar la información necesaria para continuar el procedimiento.

4.5. Ejecución de acciones de reducción

Cuando la tabla indica una reducción, representada como r_k, el algoritmo identifica la producción número k de la gramática, expresada de forma general como:

A → β

Donde A es el símbolo no terminal situado en el lado izquierdo de la producción y β es la secuencia de símbolos del lado derecho.

El procedimiento de reducción realiza las siguientes operaciones:

1. Identifica e imprime la producción que se aplicará.
2. Retira de la pila los símbolos y estados correspondientes al lado derecho de la producción.
3. Consulta el nuevo estado superior de la pila.
4. Introduce el símbolo no terminal A en la pila.
5. Consulta la transición Ir_a[j, A] para obtener el estado correspondiente.
6. Introduce el nuevo estado en la pila.

La cantidad de elementos que se retiran depende de la longitud del lado derecho de la producción. De acuerdo con el algoritmo mostrado en clase, se retiran $2 \times |\beta|$ elementos cuando la pila alterna entre símbolos y estados.

La reducción sustituye una secuencia reconocida por el símbolo no terminal que la representa, permitiendo que el análisis continúe hasta alcanzar una condición de aceptación o error.

4.6. Estados de aceptación y error

El algoritmo contempla dos condiciones principales para finalizar el análisis.

Aceptación: cuando la tabla indica la acción de aceptación, el algoritmo termina el procedimiento y comunica que la tira de tokens ha sido reconocida de acuerdo con la gramática. En el pseudocódigo proporcionado, esta acción se representa mediante la instrucción Aceptación: return.

Error: cuando la tabla no contiene una acción válida para el estado y el token actuales, el algoritmo ejecuta la operación de error. Esto indica que la entrada no puede continuar procesándose mediante las acciones definidas por la tabla.

Ambas condiciones funcionan como criterios de terminación. Mientras no se presente una aceptación o un error, el algoritmo continúa consultando la tabla y ejecutando las operaciones correspondientes.

4.7. Organización de archivos y ejecución automática

Como parte de la integración del sistema, se organizaron los componentes del proyecto en archivos separados, con el propósito de facilitar su mantenimiento, comprensión y utilización.

Se creó un archivo destinado a contener el algoritmo de análisis sintáctico, donde se concentra el procedimiento encargado de procesar la tira de tokens mediante la pila y la tabla de acciones.

También se creó un archivo para almacenar los ejemplos utilizados durante las pruebas. Esto permite disponer de entradas con las que se puede comprobar el funcionamiento del analizador y verificar su respuesta ante diferentes casos.

Finalmente, se incorporó una funcionalidad de ejecución automática que permite agregar una gramática y una tira de tokens al iniciar una ejecución. De esta manera, el programa dispone de los datos necesarios para realizar el análisis sin requerir que se introduzcan manualmente en cada prueba.

Esta funcionalidad facilita el proceso de verificación, reduce el tiempo dedicado a preparar las entradas y permite repetir las ejecuciones de manera más práctica.

5. Tecnologías utilizadas

Para el desarrollo del proyecto se utilizaron las herramientas y componentes empleados en las etapas anteriores del analizador sintáctico.

Python: lenguaje de programación utilizado para implementar la lógica del sistema y desarrollar el algoritmo encargado de procesar la entrada.

Tabla de análisis sintáctico: estructura que proporciona las acciones y transiciones necesarias para determinar qué operación debe ejecutar el algoritmo en cada paso.

Colección canónica: conjunto de estados generado durante una etapa previa del proyecto y utilizado como base para construir la tabla de análisis sintáctico.

Archivos de código y ejemplos: recursos que permiten separar la implementación del algoritmo de los datos empleados durante las pruebas.

Funcionalidad de ejecución automática: mecanismo incorporado para facilitar la preparación de las entradas y agilizar la ejecución de los ejemplos.

6. Flujo de funcionamiento

El funcionamiento general del algoritmo se puede resumir en las siguientes etapas:

1. **Preparación de la entrada:** se dispone de una gramática y una tira de tokens que se desea analizar.
2. **Inicialización:** se introduce el estado inicial en la pila y se coloca el apuntador al comienzo de la tira de entrada.
3. **Consulta:** se obtiene el estado superior de la pila y el token actual para consultar la tabla de acciones.
4. **Desplazamiento:** si la tabla indica d_j, se introducen el token y el estado correspondiente en la pila y se avanza el apuntador.
5. **Reducción:** si la tabla indica r_k, se aplica la producción correspondiente, se retiran los elementos necesarios de la pila y se consulta la transición Ir_a.
6. **Aceptación o error:** si la tabla indica aceptación, el procedimiento termina satisfactoriamente; si se presenta una acción de error, se interrumpe el análisis.
7. **Ejecución de ejemplos:** se utiliza el archivo destinado a las pruebas y la funcionalidad de carga automática para agilizar la comprobación del programa.

Este flujo permite que el algoritmo aproveche los resultados de las etapas anteriores y complete el análisis de una entrada mediante un procedimiento sistemático.

7. Conclusiones

La implementación del algoritmo de análisis sintáctico representa la etapa final del procedimiento desarrollado en el proyecto. Mediante el uso de una pila, una tira de tokens y la tabla de análisis sintáctico, el programa puede ejecutar desplazamientos y reducciones para determinar si una entrada cumple con las reglas establecidas por la gramática.

La integración de este algoritmo con la colección canónica y la tabla de análisis sintáctico permite aprovechar los resultados de las etapas anteriores y completar el proceso de reconocimiento de las entradas. De esta manera, el proyecto reúne los componentes necesarios para llevar a cabo el análisis sintáctico de acuerdo con el procedimiento estudiado en clase.

Por otra parte, la separación del algoritmo y los ejemplos en archivos distintos contribuye a mantener una organización más clara del código. La funcionalidad de ejecución automática también facilita las pruebas, ya que permite preparar una gramática y una tira de tokens sin repetir manualmente el proceso en cada ejecución.

En conjunto, estas mejoras permiten contar con un sistema más organizado y práctico para comprobar el funcionamiento del analizador sintáctico, identificar posibles errores y verificar los resultados obtenidos.

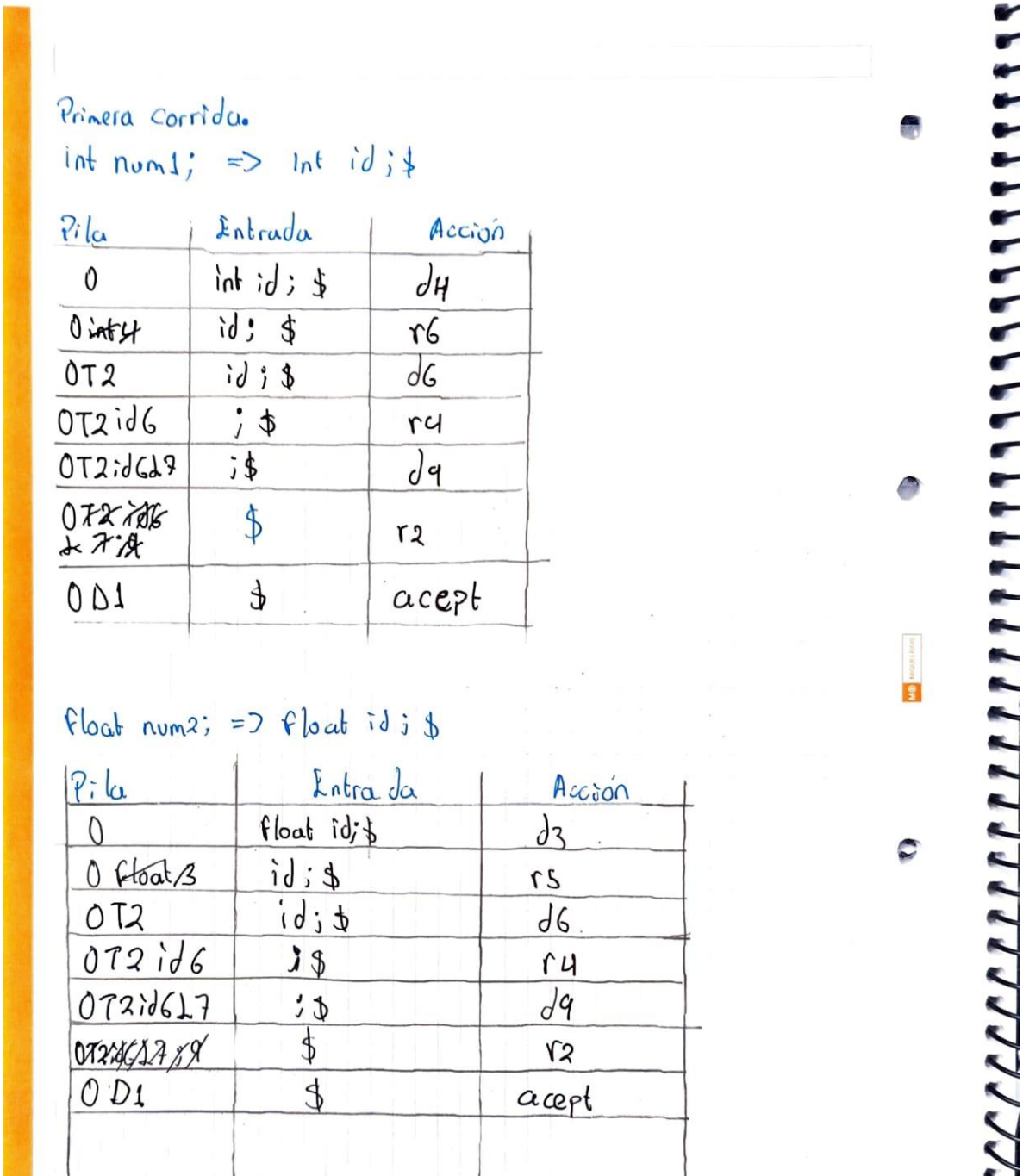
8. Asignaciones de la semana

Día	Objetivo principal	Tareas asignadas del día	Descripción específica de las actividades	Responsables

Lunes 24	Organización y análisis del proyecto	Revisar los requisitos, distribuir las actividades y preparar las gramáticas de prueba.	. Se organizaron las tareas del algoritmo del Analizador Sintáctico LR, las derivaciones correspondientes, interfaz gráfica y pruebas. También se revisaron las gramáticas que se utilizarían para comprobar los resultados.	Todo el equipo
Martes 25	Desarrollo de los algoritmos fundamentales	Implementar la lectura y validación de la tira de tokens.	Se trabajó en la lectura de los archivos de entrada. Se desarrollo el algoritmo del Analizador Sintáctico LR.	Ángel Andrés García Barela Irving Damián Álvarez Olivera Haniel López Osorio
Miércoles 26	Construcción de la colección e integración	Implementacion del algoritmo en el programa fuente e integrar la validación con la interfaz gráfica.	Se implementó el programa fuente y a través del analizador léxico, generar la Tira de tokens, que será utilizadapor el algoritmo del analizador sintáctico LR. Paralelamente, se avanzó en la conexión de los algoritmos con la interfaz para cargar gramáticas y visualizar los resultados.	Ángel Andrés García Barela Jaret Ramirez Manzano Irving Damian Alvarez Olivera
Jueves 28	Pruebas y corrección de errores	Ejecutar las pruebas g1, g2 y g3, comparar los resultados con cálculos manuales y corregir inconsistencias.	Se ejecutaron las gramáticas de prueba y se revisaron tablas obtenidas. Los resultados se contrastaron con cálculos manuales para identificar diferencias.	Miguel Angel Soriano Martinez Cesar Zaid Martinez

Viernes 29	Verificación y preparación de la entrega	Revisar el funcionamiento general, organizar los archivos de prueba y reunir las evidencias para la documentación.	Se verificó el funcionamiento integrado del programa, desde la lectura de la gramática hasta la generación de tablas. Finalmente, se organizaron los archivos de prueba, los resultados obtenidos y la documentación del proyecto para preparar la entrega.	Todo el equipo

9. Evidencias



Primera corrida.

int num1; => int id ; $

Pila	Entrada	Acción
0	int id ; $	d4
0 int4	id ; $	r6
0T2	id ; $	d6
0T2id6	; $	r4
0T2id6;7	; $	d9
0T2id6;7 $	$	r2
0D1	$	acept

float num2; => float id ; $

Pila	Entrada	Acción
0	float id;$	d3
0 float3	id ; $	r5
0T2	id ; $	d6
0T2id6	; $	r4
0T2id6;7	; $	d9
0T2id6;7 $	$	r2
0D1	$	acept

Tabla de Analisis Sintactico LR

Gramatica No. 2

D' -> D$
1: D -> DD
2: D -> T id L ;
3: L -> , id L
4; L -> ϵ
5: T -> float
6: T -> int

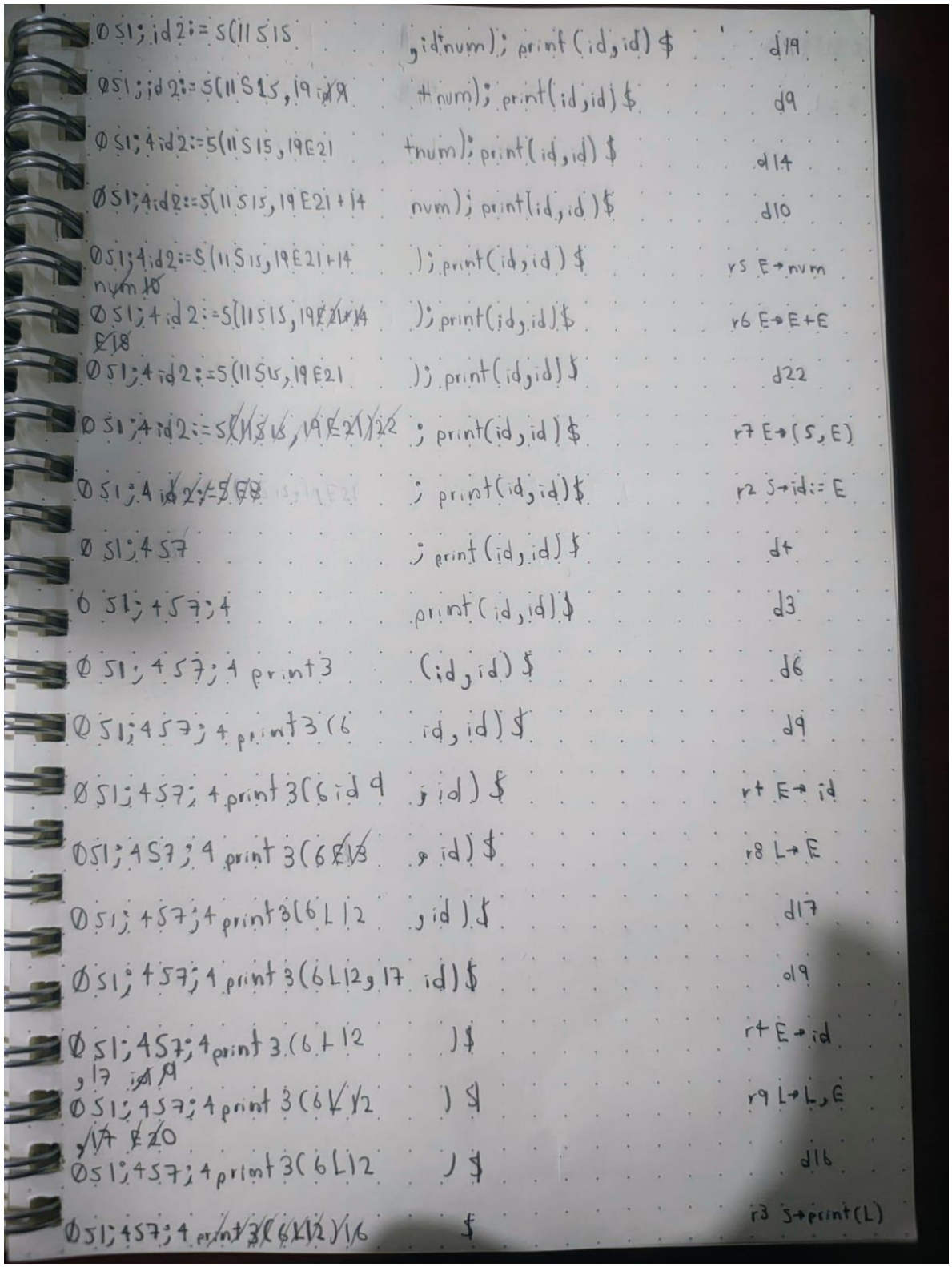
No_Terminales : D L T
Terminales : $ id ; , ϵ float int

Edos	id	;	,	float	int	$	D	L	T
0				d3	d4	accpt	1		2
1				d3	d4		5		2
2	d6								
3	r5								
4	r6								
5				d3 r1	d4 r1	r1	5		2
6		r4	d8						7
7		d9							
8	d10								
9				r2	r2	r2			
10			d8					11	
11		r3							

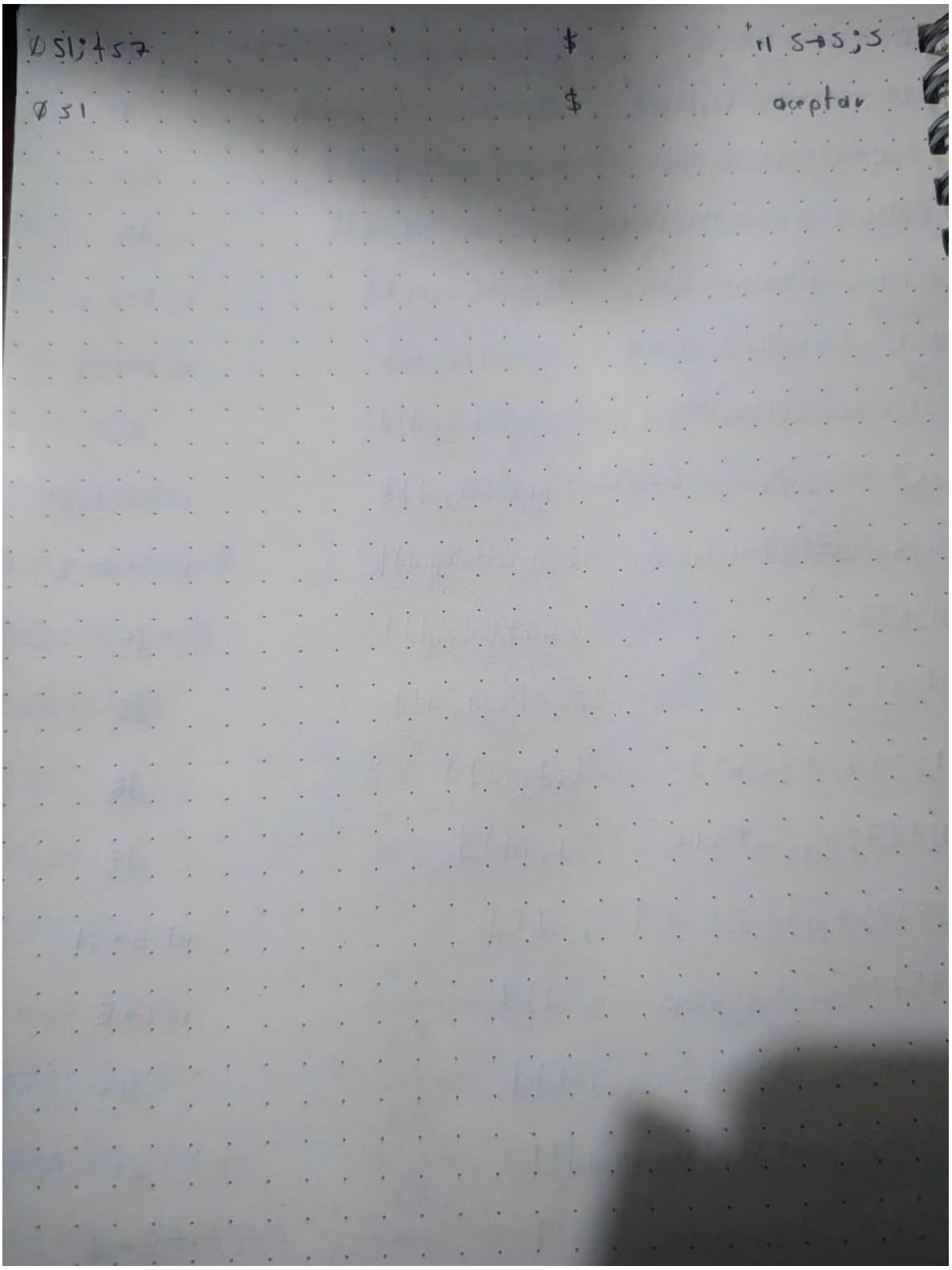
Simulacion del programa

a := 5 + 3 ; b := (a := 2 , a + 1) ; print (a , b)

Pila	Entrada	Salida
∅	id := num + num ; id := (id := num , id + num) ; print (id , id) $	d2
∅ id2	:= num + num ; id := (id := num , id + num) ; print (id , id) $	d5
∅ id2 := 5	num + num ; id := (id := num , id + num) ; print (id , id) $	d10
∅ id2 := 5 num10	+ num ; id := (id := num , id + num) ; print (id , id) $	r5 E → num
∅ id2 := 5 E8	+ num ; id := (id := num , id + num) ; print (id , id) $	d14
∅ id2 := 5 E8 + 14	num ; id := (id := num , id + num) ; print (id , id) $	d10
∅ id2 := 5 E8 + 14 num10	; id := (id := num , id + num) ; print (id , id) $	r5 E → num
∅ id2 := 5 E8 + 14 E18	; id := (id := num , id + num) ; print (id , id) $	r6 E → E + E
∅ id2 := 5 E8	; id := (id := num , id + num) ; print (id , id) $	r2 S → id := E
∅ S1	; id := (id := num , id + num) ; print (id , id) $	d4
∅ S1 ; 4	id := (id := num , id + num) ; print (id , id) $	d2
∅ S1 ; 4 id2	:= (id := num , id + num) ; print (id , id) $	d5
∅ S1 ; 4 id2 := 5	(id := num , id + num) ; print (id , id) $	d11
∅ S1 ; 4 id2 := 5 (11	id := num , id + num) ; print (id , id) $	d2
∅ S1 ; 4 id2 := 5 (11 id2	:= num , id + num) ; print (id , id) $	d5
∅ S1 ; 4 id2 := 5 (11 id2 := 5	num , id + num) ; print (id , id) $	d10
∅ S1 ; id2 := 5 (11 id2 := 5 num10	, id + num) ; print (id , id) $	r5 E → num
∅ S1 ; id2 := 5 (11 id2 := 5 E8	, id + num) ; print (id , id) $	r2 S → id := E



0 s1; id2:= s(11 s15	,id num); print(id,id) $	d19
0 s1; id2:= s(11 s15, 19 id 9	+ num); print(id,id) $	d9
0 s1; 4 id2:= s(11 s15, 19 E21	+ num); print(id,id) $	d14
0 s1; 4 id2:= s(11 s15, 19 E21 + 14	num); print(id,id) $	d10
0 s1; 4 id2:= s(11 s15, 19 E21 + 14 num 10	); print(id,id) $	r5 E→num
0 s1; 4 id2:= s(11 s15, 19 E21 + 14 E 18	); print(id,id) $	r6 E→E+E
0 s1; 4 id2:= s(11 s15, 19 E21	); print(id,id) $	d22
0 s1; 4 id2:= s(11 s15, 19 E21) 22	; print(id,id) $	r7 E→(S,E)
0 s1; 4 id 2 := s 18	; print(id,id) $	r2 S→id:= E
0 s1; 4 s7	; print(id,id) $	d4
0 s1; 4 s7; 4	print(id,id) $	d3
0 s1; 4 s7; 4 print 3	(id,id) $	d6
0 s1; 4 s7; 4 print 3 (6	id,id) $	d9
0 s1; 4 s7; 4 print 3 (6 id 9	,id) $	r4 E→id
0 s1; 4 s7; 4 print 3 (6 E 13	,id) $	r8 L→E
0 s1; 4 s7; 4 print 3 (6 L 12	,id) $	d17
0 s1; 4 s7; 4 print 3 (6 L 12, 17	id) $	d9
0 s1; 4 s7; 4 print 3 (6 L 12 , 17 id 9	) $	r4 E→id
0 s1; 4 s7; 4 print 3 (6 L 12 , 17 E 20	) $	r9 L→L,E
0 s1; 4 s7; 4 print 3 (6 L 12	) $	d16
0 s1; 4 s7; 4 print 3 (6 L 12) 16	$	r3 S→print(L)


Ø 51; 57
Ø 51
$
$
11 S→S; S
aceptar

Prueba 4

Gramatica

1º S → S ; S
2º S → id := E
3º S → print (L)
4º E → id
5º E → num
6º E → E + E
7º E → (S , E)
8º L → E
9º L → L , E

Tabla de analisis Sintactico

